

Perguntas e Respostas — LP1 (Lógica de Programação I)

| "Ser humano é inteligente e lento, IA é burra e rápida!" — Orlando

Este documento reúne as perguntas feitas pela turma ao longo das aulas de LP1, agora respondidas de forma completa e com indicação das fontes consultadas. Perguntas de tom mais descontraído da turma foram mantidas ao final, para registro da memória da turma.

1. Fundamentos de Programação

Quais as principais ferramentas de um programador?

No dia a dia, um programador costuma trabalhar com:

- Editor de código ou IDE (ex.: VS Code, PyCharm, IntelliJ) — onde o código é escrito, testado e depurado.
- Interpretador ou compilador da linguagem escolhida (ex.: o interpretador do Python).
- Sistema de controle de versão, como o Git, e plataformas como GitHub/GitLab para versionar e colaborar em projetos.
- Terminal / linha de comando, para executar programas e scripts.
- Depurador (debugger), para localizar erros passo a passo.
- Documentação oficial da linguagem e comunidades (Stack Overflow, fóruns) como fonte de consulta.

Além disso, ferramentas de gestão de projetos (Trello, Jira) e de comunicação em equipe (Slack, Discord) também fazem parte da rotina profissional.

O que é algoritmo?

Um algoritmo é uma sequência finita, ordenada e bem definida de passos que resolve um problema ou realiza uma tarefa. Ele precisa ser: *finito* (tem começo, meio e fim), *claro* (cada passo é preciso, sem ambiguidade) e *replicável* (produz o mesmo resultado sempre que executado com as mesmas entradas). Um algoritmo pode ser escrito em linguagem natural, em pseudocódigo ou, depois, traduzido para uma linguagem de programação.

Um exemplo simples do dia a dia: uma receita de bolo é um algoritmo — uma lista ordenada de passos que, seguida corretamente, sempre chega ao mesmo resultado.

O que é linguagem de programação? Cite exemplos

É um conjunto formal de regras (sintaxe) e significados (semântica) que permite ao programador escrever instruções que o computador consegue interpretar e executar. Funciona como uma ponte entre o raciocínio humano e a lógica de funcionamento da máquina. Exemplos: Python, Java, C, C++, C#, JavaScript, PHP, Ruby, Go, Rust, Kotlin, Swift, entre muitas outras.

Quais as principais linguagens de programação? Quais as diferenças entre elas?

Segundo os principais rankings de popularidade do setor (TIOBE Index, Stack Overflow Developer Survey e GitHub Octoverse), as linguagens mais usadas atualmente incluem Python, JavaScript, Java, C, C++ e C# ^[7, 8]

As diferenças mais relevantes entre elas costumam ser:

- Nível de abstração: linguagens de baixo nível (Assembly, C) trabalham próximas do hardware; linguagens de alto nível (Python, Java) abstraem detalhes de memória e são mais fáceis de ler.
- Tipagem: estática (o tipo da variável é definido e checado em tempo de compilação, como em Java e C) ou dinâmica (o tipo é resolvido em tempo de execução, como em Python).
- Execução: linguagens compiladas (C, C++) geram um executável antes de rodar; linguagens interpretadas (Python, JavaScript) são executadas linha a linha por um interpretador.
- Paradigma predominante: estruturado, orientado a objetos, funcional, ou multiparadigma.
- Área de aplicação: C/C++ para sistemas e desempenho, JavaScript para web, Python para dados/IA/automação, Java/C# para sistemas corporativos e Android.

O que é paradigma de programação?

É o estilo ou a filosofia usada para organizar e estruturar o código. Os principais são:

- Programação estruturada: o código é organizado em sequências, decisões (if/else) e repetições (loops), dividido em funções — normalmente o primeiro paradigma estudado.
- Programação orientada a objetos (POO): organiza o código em torno de "objetos" que combinam dados (atributos) e comportamentos (métodos), usando conceitos como classes, herança, encapsulamento e polimorfismo.
- Programação funcional: trata a computação como avaliação de funções matemáticas, evitando estados e dados mutáveis (ex.: Haskell, e recursos funcionais presentes em Python e JavaScript).

Como é desenvolvida uma linguagem de programação?

De forma resumida, o processo passa por: (1) definir o objetivo e o público da linguagem; (2) especificar sua sintaxe (as regras de escrita, geralmente descritas em notações formais como a BNF) e sua semântica (o significado de cada construção); (3) implementar um compilador ou interpretador, o programa que traduz o código escrito para instruções que o computador executa; (4) testar, documentar e liberar versões; e (5) construir uma comunidade e um ecossistema de bibliotecas ao redor dela. Esse processo geralmente é conduzido por uma pessoa ou um pequeno grupo inicial (como Guido van Rossum com o Python) e evolui depois com contribuições da comunidade e, muitas vezes, de comitês de padronização.

O que é um operador lógico?

Operadores lógicos são operadores que combinam ou avaliam expressões e retornam sempre um valor booleano: **verdadeiro (True) ou falso (False)**. Os três principais são E (AND), OU (OR) e NÃO (NOT). Em Python eles são escritos como palavras (and, or, not); em outras linguagens, como C, Java e JavaScript, costumam aparecer como símbolos (&, |, !). Eles são a base para tomar decisões em um programa (estruturas condicionais) e também são usados fora da programação, na lógica proposicional clássica.

O que é código-fonte?

É o texto do programa escrito pelo programador em uma linguagem de programação, antes de ser processado pelo computador. É o código-fonte que fica em arquivos como .py, .java ou .c e que é lido por um interpretador ou transformado em código executável por um compilador.

O que é uma variável?

É um espaço na memória do computador, identificado por um nome, usado para armazenar um valor que pode ser lido e alterado durante a execução do programa. Por exemplo, em Python, `idade = 20` cria uma variável chamada `idade` que guarda o valor 20, e esse valor pode mudar depois.

Qual o conceito de programação?

Programar é o processo de escrever, em uma linguagem que o computador entende, uma sequência de instruções (um algoritmo) para que a máquina realize uma tarefa específica — desde um cálculo simples até sistemas complexos como aplicativos, sites e modelos de inteligência artificial.

Como se começa um código?

De forma geral, o processo é:

- Entender bem o problema antes de escrever qualquer linha de código.
- Rascunhar a lógica em pseudocódigo ou num fluxograma.
- Configurar o ambiente (instalar a linguagem, abrir um editor).
- Escrever o programa em pequenos passos, testando cada parte.
- Testar com diferentes entradas e corrigir os erros encontrados.

Quais os erros mais comuns de quem está começando?

O medo de errar costuma ser o maior obstáculo — errar faz parte do processo normal de aprender a programar, já que o próprio interpretador aponta onde está o problema. Outros erros comuns:

- Pular os fundamentos de lógica para ir direto a projetos complexos.
- Copiar código pronto sem entender o que ele faz.
- Não ler as mensagens de erro com atenção.
- Não praticar com regularidade (programação é uma habilidade que se constrói com repetição).
- Não usar controle de versão desde o início.

Boas práticas para programar? / Quais são boas práticas de programação?

- Usar nomes de variáveis e funções descritivos.
- Manter funções pequenas e com uma única responsabilidade.
- Comentar o código quando necessário, sem exagerar.
- Seguir um guia de estilo (em Python, o PEP 8).
- Testar o código com frequência, inclusive com testes automatizados.
- Versionar o código com Git desde o primeiro projeto.

Como praticar programação?

- Resolver exercícios em plataformas como HackerRank, Codewars, LeetCode ou Beecrowd.
- Construir pequenos projetos pessoais que resolvam problemas reais.
- Contribuir com projetos de código aberto no GitHub.
- Refazer, do zero, projetos vistos em aula, sem consultar a solução original.

O que é HTML5?

HTML5 (HyperText Markup Language, versão 5) é a versão mais recente e consolidada da linguagem de marcação usada para estruturar o conteúdo de páginas web — títulos, parágrafos, imagens,

formulários, vídeos etc. Tecnicamente, HTML não é uma linguagem de programação (não possui lógica de decisão ou repetição própria): é uma linguagem de marcação, que descreve a estrutura de um documento e é interpretada pelo navegador junto com CSS (estilo) e JavaScript (comportamento/lógica).

2. Python

O que é Python?

Python é uma linguagem de programação de alto nível, interpretada, multiparadigma e de tipagem dinâmica, criada por Guido van Rossum e lançada em 1991. É conhecida pela sintaxe simples e legível, próxima da linguagem natural, o que reduz a curva de aprendizado. Hoje é uma das linguagens mais populares do mundo, muito usada em automação, desenvolvimento web (back-end), ciência de dados, e especialmente em inteligência artificial e machine learning ^[7, 8, 9]

Por que vamos usar Python e não outra linguagem?

Python costuma ser a primeira linguagem escolhida em cursos de lógica de programação por alguns motivos:

- Sintaxe simples e próxima do inglês/português, o que deixa o foco na lógica e não na sintaxe.
- Tipagem dinâmica: não é preciso declarar o tipo da variável nem gerenciar memória manualmente.
- Grande quantidade de bibliotecas prontas para praticamente qualquer área.
- Comunidade enorme e ativa, o que facilita encontrar materiais e tirar dúvidas.
- É, hoje, a linguagem mais demandada em áreas de IA e ciência de dados, o que dá ao aprendizado aplicação direta no mercado.

O foco da nossa aula será apenas Python ou veremos outra linguagem de programação?

Nesta disciplina (LP1), o foco é usar Python como ferramenta para aprender lógica de programação — variáveis, estruturas condicionais, repetição, funções etc. Esses conceitos, porém, são transferíveis: uma vez entendida a lógica, migrar para outra linguagem (Java, C, JavaScript...) é sobretudo uma questão de aprender uma nova sintaxe.

Python ou pyton?

O nome correto da linguagem é Python. "Pyton" (com um só "h" a menos) não existe como linguagem — e, curiosamente, "píton" em português é o nome popular da cobra. Vale um dado curioso: apesar do símbolo da linguagem ser uma cobra, o nome Python foi escolhido por Guido van Rossum em homenagem ao grupo de humor britânico Monty Python's Flying Circus, do qual ele era fã — não em referência ao réptil.

Python é sonserina? (Cobra)

Sonserina não! É só uma brincadeira com o fato de o símbolo do Python ser uma cobra (o logo mostra, inclusive, duas cobras entrelaçadas). Como visto na pergunta anterior, o nome vem do grupo de comédia Monty Python, sem relação com a casa de Hogwarts.

Livros para estudar Python?

- Documentação oficial do Python (python.org/doc) — gratuita e sempre atualizada.

- "Automate the Boring Stuff with Python", de Al Sweigart — disponível gratuitamente online em automatetheboringstuff.com, focado em projetos práticos.
- "Python Fluente" (Fluent Python), de Luciano Ramalho — referência para quem já tem alguma base e quer aprofundar.
- "Curso em Vídeo" (Python), de Gustavo Guanabara — conteúdo gratuito em português, muito usado por iniciantes no Brasil.
- W3Schools e MDN Web Docs — boas referências rápidas de sintaxe e exemplos.

Tem algum link de livros para baixar PDF grátis?

Vale um alerta importante: sites como Library Genesis (LibGen), Anna's Archive e Z-Library disponibilizam livros sem autorização dos autores e editoras, o que constitui violação de direitos autorais — por isso, não é recomendável indicá-los ou utilizá-los. Para estudar de graça e de forma legal, as melhores opções são:

[Coursera](#) — cursos de universidades e empresas parceiras; muitos podem ser feitos gratuitamente no modo "auditoria" (só o certificado é pago).

[edX](#) — plataforma com cursos de universidades como Harvard e MIT, também com opção gratuita de acompanhamento.

Além disso: [freeCodeCamp](#) (cursos práticos e gratuitos, com certificado), a [documentação oficial do Python](#) (gratuita), e o livro [Automate the Boring Stuff with Python](#), que o próprio autor disponibiliza de graça e legalmente online.

3. Carreira em Programação / TI

O que posso criar sabendo programar?

Depende da área e das ferramentas aprendidas, mas, de forma geral, quem programa pode criar: sites e aplicações web; aplicativos para celular; jogos; sistemas de automação de tarefas repetitivas (scripts); ferramentas de análise de dados e dashboards; modelos de inteligência artificial e machine learning; APIs que conectam diferentes sistemas; e até hardware programável (robótica, IoT), entre muitas outras possibilidades.

Como se tornar um programador "top"?

- Construir uma base sólida de lógica e fundamentos, em vez de pular direto para frameworks.
- Praticar constantemente com projetos reais, não só exercícios teóricos.
- Montar um portfólio (por exemplo, no GitHub) que mostre esses projetos.
- Aprender a ler e a debugar código de outras pessoas, não só o próprio.
- Desenvolver inglês técnico, já que a maior parte da documentação e das comunidades usa esse idioma.
- Cultivar soft skills: comunicação, trabalho em equipe e resolução de problemas.

Para começar na área de TI, é bom também fazer uma pós-graduação?

Não é um pré-requisito. A entrada na área costuma ocorrer por diferentes caminhos — graduação, cursos técnicos, bootcamps intensivos ou autodidatismo com portfólio — e o que mais pesa nas contratações costuma ser a capacidade prática de resolver problemas. Uma pós-graduação tende a fazer mais sentido depois de alguma experiência, quando o objetivo é se especializar em uma área

específica (ex.: segurança da informação, ciência de dados, arquitetura de software) ou seguir carreira acadêmica.

Como e onde estudar programação em casa?

- Plataformas gratuitas: Coursera, edX, freeCodeCamp, Khan Academy, documentação oficial das linguagens.
- Canais e cursos em português, como o Curso em Vídeo.
- Plataformas de exercícios: HackerRank, Codewars, Beecrowd, Exercism.
- Comunidades: fóruns, Discord de programação e Stack Overflow para tirar dúvidas.
- O mais importante: praticar escrevendo código todos os dias, mesmo que pouco tempo por vez.

Qual a média salarial de um programador?

Os valores variam bastante conforme a fonte, o nível de experiência, a especialidade e a região do Brasil. Segundo levantamentos de 2026: o salário médio de um Programador fica em torno de **R\$ 4.576,24**^[1], enquanto outro levantamento situa a faixa de piso e teto para "Programador de Computador" entre **R\$ 5.670,48 e R\$ 11.683,68**^[2]. Para quem está começando, a faixa fica geralmente entre **R\$ 3.500,00 e R\$ 6.000,00** mensais, podendo ser maior em fintechs ou para quem domina tecnologias específicas e inglês^[3]. Fatores como stack tecnológica, modelo de contratação (CLT ou PJ), região e domínio do inglês influenciam bastante o valor final.

4. Programação, Inteligência Artificial e Sociedade

Como o uso de inteligência artificial consome água?

As IAs não consomem água diretamente — o gasto está ligado ao *resfriamento dos data centers* que processam os modelos. Quando os servidores respondem a comandos, milhares de cálculos são feitos, o que gera calor; esse calor precisa ser dissipado, e boa parte dos data centers ainda usa água em sistemas de resfriamento (torres evaporativas ou ar-condicionado)^[4]. Estimativas apontam que cada comando de texto enviado a um chatbot pode gastar entre 10 e 25 ml de água, e a geração de uma imagem pode multiplicar esse valor em até 30 vezes^[5, 6]. Em escala, isso se torna relevante: só nos data centers dos Estados Unidos, o consumo de água em 2025 chegou perto de 1 trilhão de litros, valor comparável ao consumo anual da cidade de Nova York^[6]. Por isso, empresas como Google, Amazon e Microsoft vêm investindo em sistemas de resfriamento por líquido ou imersão, que reduzem bastante essa necessidade de água^[6].

Um carro autônomo (Uber, por exemplo) está em iminência de um acidente e precisa escolher entre atropelar um animal ou frear sob risco de machucar o passageiro — como se dá a programação e a linguagem para decidir nesse caso?

Esse é um exemplo do chamado "problema do bonde" (trolley problem), um dilema clássico de ética aplicado à programação de veículos autônomos, área conhecida como ética das máquinas (machine ethics). Na prática, esse tipo de decisão não é resolvida com uma única regra simples de if/else; envolve algumas camadas:

- Sensoriamento e predição: sensores (câmeras, radar, lidar) e modelos de machine learning estimam a posição, velocidade e o risco de cada objeto ao redor do veículo.
- Regras de segurança e prioridades: os sistemas geralmente são programados para minimizar o risco de dano a pessoas antes de qualquer outra prioridade, seguindo normas de

engenharia de segurança (como a norma internacional ISO 26262, sobre segurança funcional em veículos).

- Não há consenso ético universal: diferentes abordagens filosóficas (utilitarista — minimizar o dano total; ou baseada em regras/deontológica — nunca violar certos princípios, como não desviar deliberadamente para atingir alguém) levam a decisões de programação diferentes, e isso ainda é debatido por pesquisadores, engenheiros e legisladores.

Na prática, fabricantes tendem a programar os veículos para agir de forma previsível e priorizar frenagem/redução de dano em vez de manobras evasivas complexas, já que cenários de "dilema puro" como o do enunciado são raros — a maior parte da programação é dedicada a evitar que a situação de risco chegue a existir.

5. Perguntas descontraídas da turma

Algumas perguntas fizeram parte do clima leve da turma e foram mantidas aqui como registro:

Até que horas vai a aula?

12:59 😞 (resposta original da turma, mantida por carinho).

"Vamos fazer lobbying para nós aprendermos a programar em assembly"

Fica registrado o pedido da turma! Assembly é uma linguagem de baixíssimo nível, muito próxima das instruções do processador, e costuma ser vista em disciplinas de arquitetura de computadores, mais à frente do curso — mas é, sem dúvida, um excelente exercício para entender o que acontece "por baixo" de linguagens de alto nível como Python.

Referências

[1] Quanto ganha um Programador? Salário atualizado em 2026 — Quero Educação — <https://querobolsa.com.br/cursos-e-faculdades/sistemas-de-informacao/quanto-ganha-programador-salario>

[2] Programador de Computador — Salário 2026, Piso Salarial — Portal Salário — <https://www.salario.com.br/profissao/programador-de-computador-cbo-317110/>

[3] Quanto ganha um programador iniciante no Brasil em 2026? — GrowDev — <https://growdev.com.br/quanto-ganha-programador-iniciante/>

[4] Como o uso de inteligências artificiais consome água? — UFSM — <https://www.ufsm.br/2025/09/04/como-o-uso-de-inteligencias-artificiais-consome-agua>

[5] Usar IA gasta água? Entenda o que são os data centers e como funcionam — Fast Company Brasil — <https://fastcompanybrasil.com/impacto/usar-ia-gasta-agua-entenda/>

[6] Datacenters, água e energia: o ponto de inflexão da era da IA — Canal Solar — <https://canalsolar.com.br/datacenters-agua-energia-inflexao-era-ia/>

[7] As 5 principais linguagens de programação para dominar em 2026 — Canaltech — <https://canaltech.com.br/mercado/as-5-principais-linguagens-de-programacao-para-dominar-em-2026/>

[8] As 10 linguagens de programação mais utilizadas em 2026 — OpServices — <https://www.opservices.com.br/linguagens-de-programacao-mais-utilizadas-2026/>

- [9] Linguagem de programação mais usada em 2026: guia completo — Mestres da Web — <https://www.mestresdawebsite.com.br/tecnologias/linguagem-de-programacao-mais-usada-qual-dominar>
- [—] Coursera — <https://www.coursera.org/>
- [—] edX — <https://www.edx.org/>
- [—] freeCodeCamp — <https://www.freecodecamp.org/>
- [—] Documentação oficial do Python — <https://docs.python.org/pt-br/3/>
- [—] Automate the Boring Stuff with Python (Al Sweigart) — <https://automatetheboringstuff.com/>